

Computer Science COMP-3220 – Summer 2026

Assignment 3

Due: End of Sunday, July 26, 2026

Note: Submit your assignment as a **single pdf file** on Brightspace. The pdf file should be named Assignment3_StudentId.pdf, which replaces *StudentId* with your university student number.

Total Mark: 40

Question 1 — Adapter and Facade Patterns (13 marks)

Modernizing a Smart Home System

A company has purchased a third-party smart light system. The existing application expects all devices to implement the following interface:

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}
```

However, the third-party library provides:

```
class LegacyLight {  
  
    public void powerOn() {  
        System.out.println("Light ON");  
    }  
  
    public void powerOff() {  
        System.out.println("Light OFF");  
    }  
}
```

In addition, users currently need to manually control multiple home theater components:

```
DVDPlayer dvd = new DVDPlayer();  
Projector projector = new Projector();  
SoundSystem sound = new SoundSystem();  
  
dvd.on();  
projector.on();  
sound.on();
```

Tasks:

1. Complete the design using the **Adapter Pattern**. (5 marks)
 - o Create the SmartDevice interface.
 - o Implement a LightAdapter.
 - o Allow the client to use LegacyLight through SmartDevice.
2. Complete the design using the **Facade Pattern**. (4 marks)
 - o Create a HomeTheaterFacade class.
 - o Add a method watchMovie().
 - o Simplify the client interaction.

3. Explain:
 - When Adapter should be preferred over Facade. (2 marks)
4. Draw UML class diagrams for both solutions. Include: (2 marks)
 - Adapter, Target, Adaptee
 - Facade and subsystem classes

Question 2 — Template Method Pattern (13 marks)

Building a Report Generation Framework

A company generates reports in multiple formats.
The overall workflow is always:

1. Read data
2. Process data
3. Export report

Currently, developers duplicate code:

```
class PDFReport {

    public void generate() {

        readData();
        processData();
        exportPDF();
    }
}

class ExcelReport {

    public void generate() {

        readData();
        processData();
        exportExcel();
    }
}
```

Tasks:

1. Refactor the design using the **Template Method Pattern**. (6 marks)

Complete:

- Abstract class ReportGenerator
- Template method generateReport()
- Abstract export() method

2. Implement:

- PDFReportGenerator
- ExcelReportGenerator

(2 marks)

3. Add a hook method allowing subclasses to optionally perform validation before exporting. (2 marks)
4. Draw a UML class diagram. Include: (3 marks)

- Abstract class
- Template method
- Hook method
- Concrete subclasses

Question 3 — Iterator and Composite Patterns (14 marks)

Designing a Company Organization Structure

A company wants to represent its organization hierarchy.

Departments may contain:

- Employees
- Other Departments

The company also wants to traverse the entire hierarchy and print all employee names.

You are given:

```
interface OrganizationComponent {
    void display();
}
class Employee implements OrganizationComponent {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public void display() {
        System.out.println(name);
    }
}
```

Tasks:

1. Complete the Composite Pattern implementation. (6 marks)
Create:
 - Department class
 - add()
 - remove()
 - display()
2. Implement an Iterator that traverses all components inside a Department. (4 marks)
3. Explain why Composite and Iterator work well together in this scenario. (1 mark)
4. Draw a UML class diagram showing: (3 marks)
 - Component
 - Leaf
 - Composite
 - Iterator